

Geometry under anchoring: a whole-geometry read over the stored runs

A reanalysis of 131 stored checkpoints from three experiments. Past the first block, an anchored model does not arrange its colours the way an un-anchored one does. By the last block, the colour geometry of an anchored run correlates with that of a control at half the level two controls reach, in every experiment. Dropping the anchor axis brings the recipe at 50 epochs back to the control band. The heavier anchors, the longer-trained recipe, and the handover arms stay under it, and their seeds agree with each other more closely than controls do. So a light anchor seems to add an axis to a control's geometry; a heavier or longer one also moves the deep geometry to a different arrangement, one that seeds reproduce and one less like the RGB cube.

Observations

Each line below is a read on stored checkpoints, measured against the spread between un-anchored controls. None of them is a result; the closing section says what a preregistered follow-up would test.

- **Whole geometry:** at the embedding, anchored runs are about as close to a control as controls are to each other. The gap opens with depth. At the last block, the colour geometry of an anchored run correlates with that of a control at roughly half the control-against-control level. That holds for the D2.1 recipe, the six-op recipe at both lengths, the three heavier survey points, and all three handover arms.
- **The anchor axis:** with e_1 dropped from every run, the six-op recipe at 50 epochs comes back inside the control band at the last two blocks. The D2.1 recipe and the two mid-weight survey points come back most of the way, the 100-epoch recipe and the heaviest point about a third of the way, and the handover arms barely move. So what differs in those last cases is the geometry of the other 63 coordinates. At the embedding, the same read moves every anchored condition a little *under* the band, because it removes *red* from the anchored side only.
- **Agreement among seeds:** at depth, anchored runs of one condition agree with each other more closely than controls do. With e_1 dropped, the extra agreement stays for the heavier anchors, the 100-epoch recipe, and the handover arms. It goes away for the $\lambda=0.1$ recipes at 50 epochs, which are the conditions the axis read put back in the band. So where the geometry differs beyond the axis, seeds reproduce that difference.
- **Dose:** in the full read, the drop at the last block is a step at $\lambda=0.1$, with little further change up to $\lambda=0.557$. With e_1 dropped, the heavier anchors keep less of the control's geometry than the recipe does.
- **The colour cube:** at the last two blocks, anchored runs arrange the colours less like the RGB cube than controls do. The embedding and first block are alike.
- **Procrustes** agrees with RSA on every read, so a rotation and a rescaling would not remove the difference.

Scope

This is a reanalysis of stored checkpoints, planned in the [backlog item](#):

no training, no gates, no verdicts. Every anchored model we have has already been scored for the axis it was given (alignment, margin, containment) and for its task. But whether the *rest* of its colour geometry matches what an un-anchored model builds has only been read through per-channel probe R^2 (the H2 of ex-2.1.12), which came back unresolved. Here we ask that of the geometry as a whole.

Why

The D2.1 post claims that anchoring guides one concept to a known place and leaves the rest of the representation to form as it would have. That claim rests on the task metric (an anchored model still answers as well as a control) and on probes for the other channels.

Neither one looks at the geometry itself. A model can score the same while arranging its colours differently, and a linear probe can find green wherever it is put. So the claim needs a statistic that compares the *shape* of the representation between an anchored run and an un-anchored one, with the ordinary variation in that shape between un-anchored seeds as the yardstick.

Representational similarity analysis (RSA) is that statistic.¹ For each run it takes the distance between every pair of the 216 grid colours, giving a matrix of distances; it then correlates the matrices of two runs. Two runs that place the colours the same way up to a rotation correlate at 1, whatever basis each of them chose; runs that arrange them differently score lower. The baseline is control against control: runs that differ only by seed set the correlation a faithful anchored run should reach.

Anchoring is meant to move something: *red* is asked to lie along the e_1 axis. So a second variant of every read drops e_1 from the states of every run before the distances are taken, anchored and control alike. That variant asks whether the geometry *other than the anchor axis* is what a control builds.

The runs read

Every run comes from a checkpoint a published experiment left in the store. Three experiments, four groups, each with its own un-anchored control seeds.

experiment	condition	group	λ	seeds	what it is
ex-2.1.10	lam0	main	0	3	un-anchored
ex-2.1.10	either-t100	main	0.1	9	the D2.1 recipe
ex-2.2.3	control-short	main	0	5	un-anchored, 50 epochs
ex-2.2.3	recipe-short	main	0.1	20	the recipe, 50 epochs
ex-2.2.3	t12	main	0.361	5	survey proposal t12
ex-2.2.3	t48	main	0.431	5	survey proposal t48
ex-2.2.3	t00	main	0.557	5	survey proposal t00
ex-2.2.3	control	long	0	5	un-anchored, 100 epochs
ex-2.2.3	recipe	long	0.1	20	the recipe, 100 epochs
ex-2.2.9	control	main	0	5	un-anchored
ex-2.2.9	handover	main	0.1	20	untied readout, whole-line labeller
ex-2.2.9	handover-slot	main	0.1	20	untied readout, slot labeller
ex-2.2.9	handover-tied	main	0.1	9	tied readout, whole-line labeller

The runs read. Every seed of each condition is one checkpoint from the store. Conditions in one group share the control they are read against. ex-2.2.3's `recipe-short` and `recipe` include the fifteen addendum seeds each; ex-2.2.9's `handover-tied` has nine.

The read

For each run, the residual state at one site is a 216×64 matrix, one row per grid colour. At *operand 1*, the state above the first token depends on that token alone, since attention is causal, so it is the context-free representation of the colour. At *operand 2*, the state depends on the whole prompt so far, and we average it over every first operand under the reference op (`mix`, or `+` on the D2.1 grammar). Both sites are read at all five residual slices.

Each figure below shows, per condition and slice, a column of seed dots: the mean RSA of each run to the controls of its group. For a control run, that mean is taken over the *other* controls. The grey strip behind each slice is the range of control-against-control pairs, so an anchored condition whose dots sit in the strip is as close to a control as controls are to each other.

The lower row of each figure drops e_1 , the anchor axis, from the states of every run first. The [checks](#) below say how much of the variance of each run that coordinate accounts for, and whether it is where a probe reads *red* from.

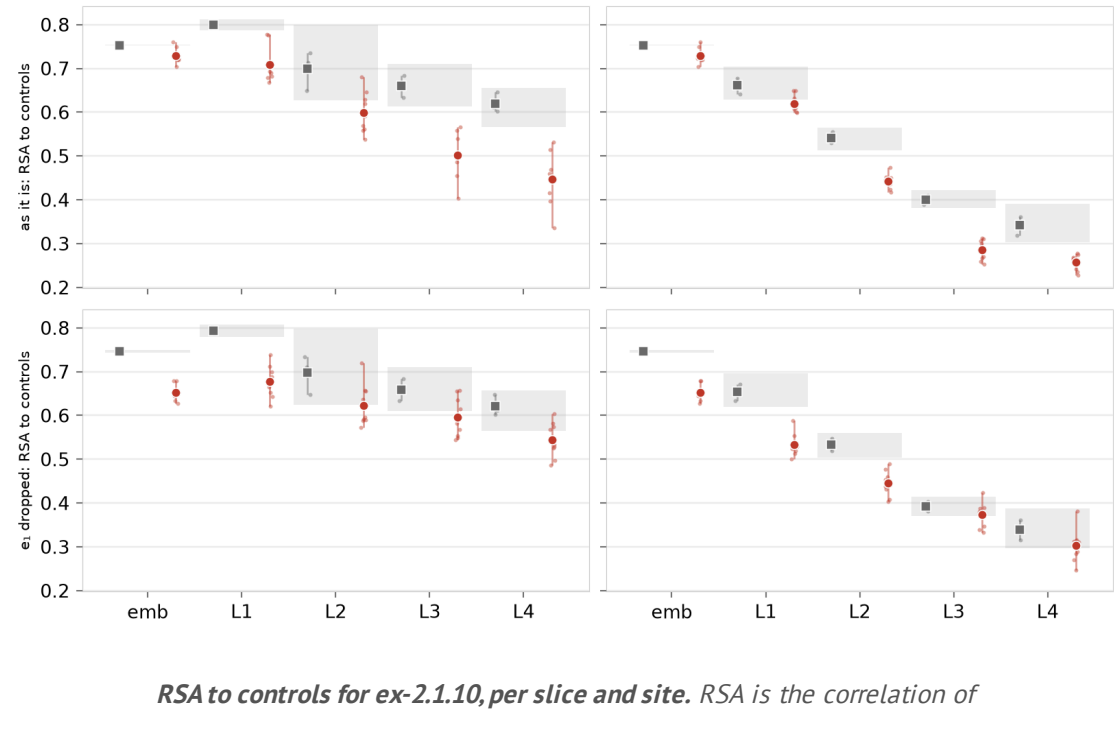
Whole geometry, per experiment

What we expected. If anchoring only moves *red* and leaves the rest alone, an anchored run should sit in the control band once *e*₁ is dropped, at every slice, and near it before.

What we saw. At the embedding, every anchored condition is within a few hundredths of the control band. The gap then opens block by block. At the last block, every anchored condition sits well under the band at both sites: the D2.1 recipe, the six-op recipe at either length, the three heavier survey points, and all three handover arms.

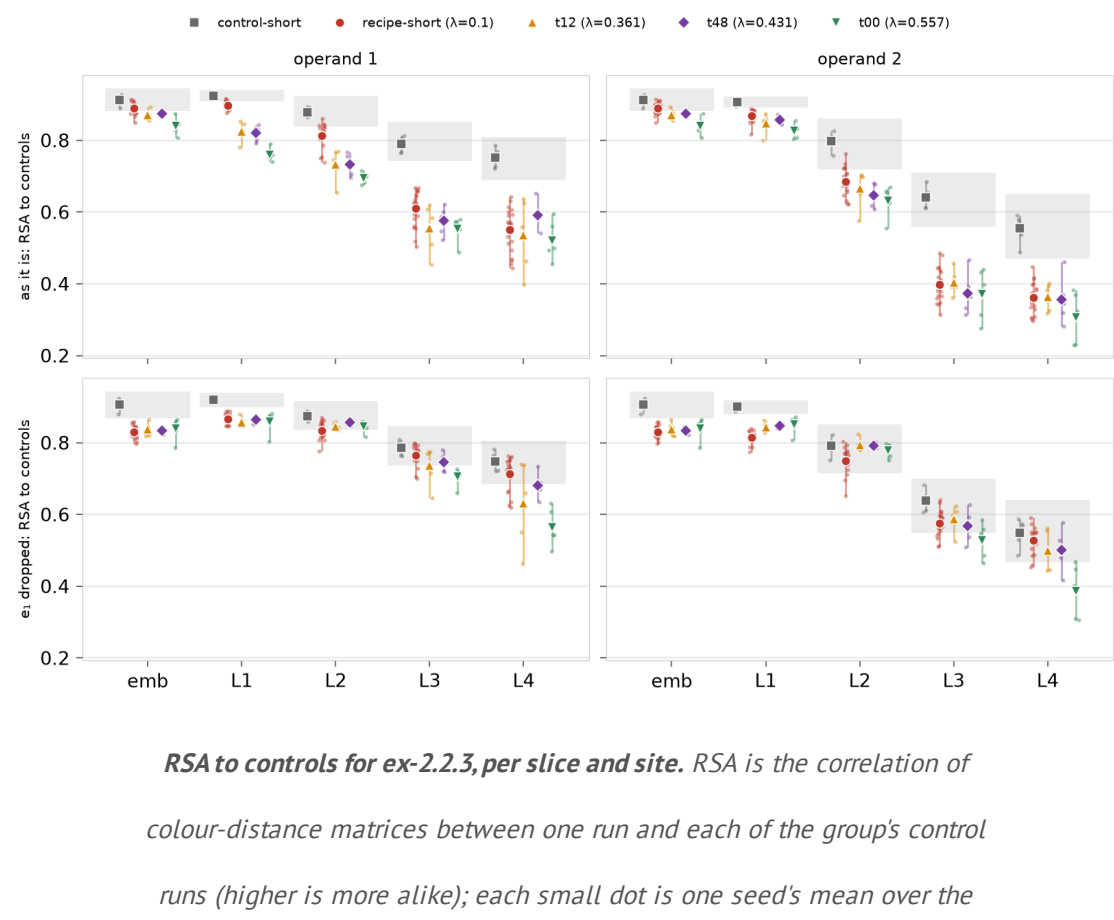
Dropping *e*₁ closes the gap for the six-op recipe at 50 epochs, which returns to the band at the last two blocks. It closes most of the gap for the D2.1 recipe and the two mid-weight survey points, about a third of it for the 100-epoch recipe and the heaviest point, and almost none on the handover grammar, where the three arms stay far under the band even with *e*₁ gone.

At the embedding, the *e*₁-dropped read goes the other way: every anchored condition falls a little under the band. That is what we would expect, since *e*₁ holds *red* for an anchored run and nothing in particular for a control, so the read removes red from one side of the comparison only. The deeper blocks come back despite that handicap.



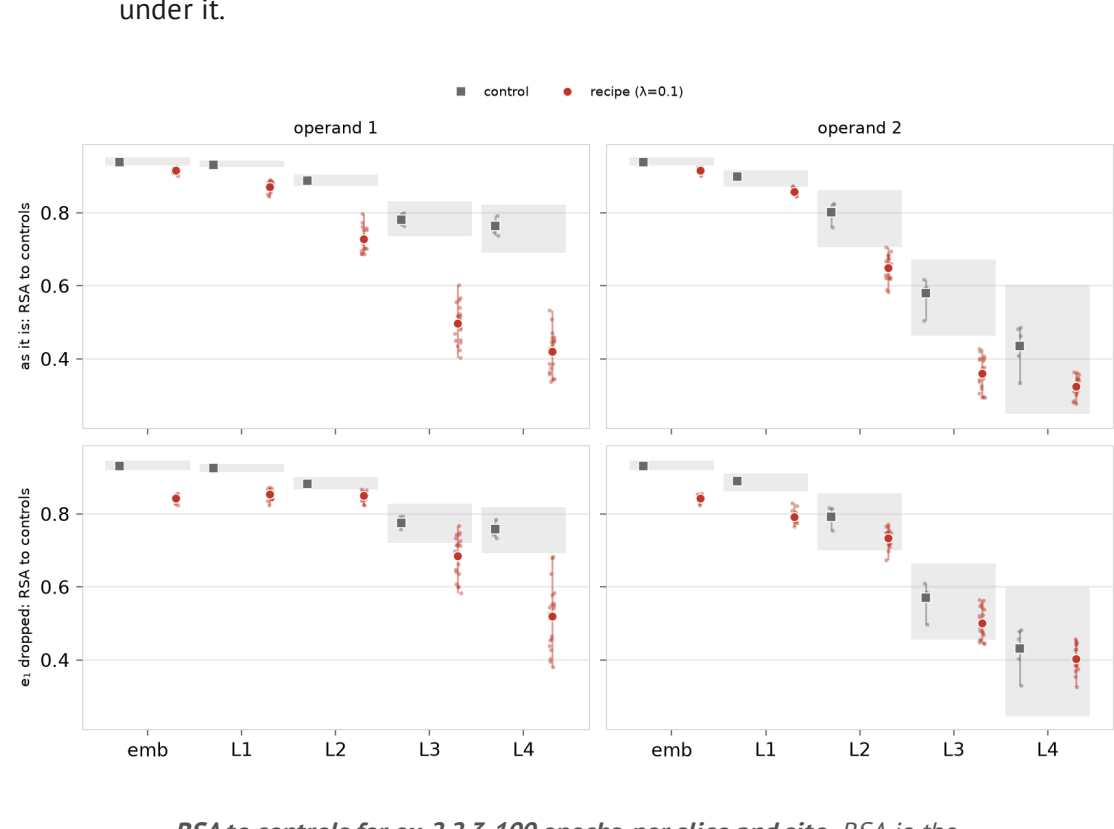
RSA to controls for ex-2.1.10, per slice and site. *RSA is the correlation of colour-distance matrices between one run and each of the group's control runs (higher is more alike); each small dot is one seed's mean over the controls, the larger mark the seed mean, and the bar the seed range. The grey strip is the range of control-against-control pairs. Top row: the geometry as it is. Bottom row: the anchor axis *e*₁ dropped from every run first.*

ex-2.1.10 has three control seeds, so the band is only three pairs wide and should be read loosely. The recipe falls under the band from the second block on. Dropping *e*₁ recovers most of the gap at the last two blocks, to within a few hundredths of the band.



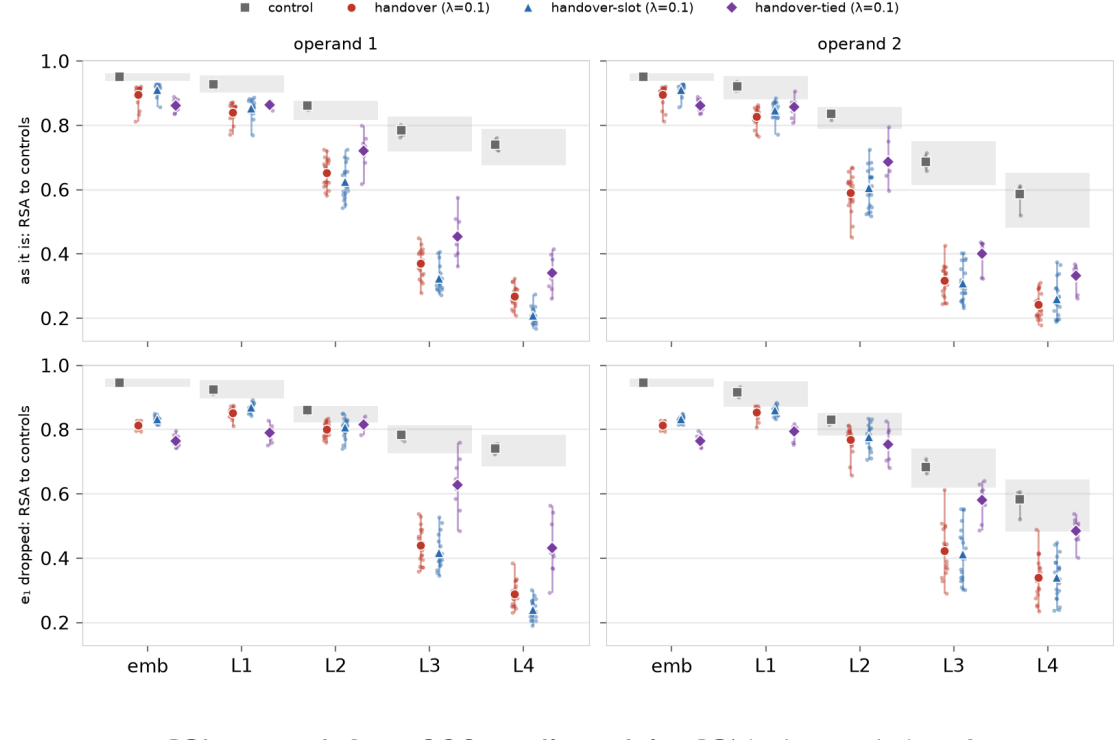
RSA to controls for ex-2.2.3, per slice and site. *RSA is the correlation of colour-distance matrices between one run and each of the group's control runs (higher is more alike); each small dot is one seed's mean over the controls, the larger mark the seed mean, and the bar the seed range. The grey strip is the range of control-against-control pairs. Top row: the geometry as it is. Bottom row: the anchor axis *e*₁ dropped from every run first.*

The short conditions of ex-2.2.3 are the clearest case for the anchor axis. At the last two blocks, the recipe is back inside the band once *e*₁ is dropped, *t*₄₈ nearly so, and *t*₁₂ most of the way. *t*₀₀ stays well under it.



RSA to controls for ex-2.2.3, 100 epochs, per slice and site. *RSA is the correlation of colour-distance matrices between one run and each of the group's control runs (higher is more alike); each small dot is one seed's mean over the controls, the larger mark the seed mean, and the bar the seed range. The grey strip is the range of control-against-control pairs. Top row: the geometry as it is. Bottom row: the anchor axis *e*₁ dropped from every run first.*

The same recipe trained for 100 epochs sits further under its band than the 50-epoch one, and dropping *e*₁ recovers less of the gap. So the difference seems to grow with training.



RSA to controls for ex-2.2.9, per slice and site. *RSA is the correlation of colour-distance matrices between one run and each of the group's control runs (higher is more alike); each small dot is one seed's mean over the controls, the larger mark the seed mean, and the bar the seed range. The grey strip is the range of control-against-control pairs. Top row: the geometry as it is. Bottom row: the anchor axis *e*₁ dropped from every run first.*

On the handover grammar, every arm is far under the band at the last two blocks, and dropping *e*₁ changes little. The tied readout (*handover-tied*) keeps the most, the slot labeller the least. We read the arms at operand 1 under *mix*. The control shares the eleven-op grammar and stochastic rounding with them, so the grammar itself is not what makes the difference.

	condition	as it is: control band	to within controls		e1 dropped: control band	to within controls	
ex-2.1.10	lam0	0.57–0.66	0.62	0.62	0.56–0.66	0.62	0.62
	either-t100	0.57–0.66	0.45	0.77	0.56–0.66	0.54	0.55
ex-2.2.3	control-short	0.69–0.81	0.75	0.75	0.68–0.81	0.75	0.75
	recipe-short	0.69–0.81	0.55	0.88	0.68–0.81	0.71	0.74
	t12	0.69–0.81	0.54	0.87	0.68–0.81	0.63	0.78
	t48	0.69–0.81	0.59	0.87	0.68–0.81	0.68	0.82
	t00	0.69–0.81	0.52	0.87	0.68–0.81	0.56	0.85
ex-2.2.3	control	0.69–0.82	0.76	0.76	0.69–0.82	0.76	0.76
	recipe	0.69–0.82	0.42	0.92	0.69–0.82	0.52	0.86
ex-2.2.9	control	0.68–0.79	0.74	0.74	0.69–0.78	0.74	0.74
	handover	0.68–0.79	0.27	0.89	0.69–0.78	0.29	0.89
	handover-slot	0.68–0.79	0.21	0.93	0.69–0.78	0.24	0.92
	handover-tied	0.68–0.79	0.34	0.93	0.69–0.78	0.43	0.88

RSA at the last block, operand 1. *Per condition: the range of control-against-control pairs, the seed mean of each run's RSA to the controls, and the mean over every pair of runs within the condition, for the geometry as it is and with *e*₁ dropped. A control's to controls leaves each run out of its own comparison, so its two columns are the same number.*

Agreement among seeds

What we expected. A run whose deep geometry is far from every control could get there two ways. Either anchoring adds seed-to-seed variation, in which case the run is far from every other run as well; or anchoring replaces one arrangement with another, in which case the run is close to the other runs of its own condition. Comparing runs within a condition tells the two apart.

What we saw. The second one, in the full read. Past the first block, anchored runs of one condition agree with each other more closely than controls do, in every group. The agreement among controls falls with depth, while the agreement within an anchored condition holds.

With e_1 dropped, that extra agreement stays for the heavier anchors, the 100-epoch recipe, and the handover arms. It goes away for the two $\lambda=0.1$ recipes at 50 epochs; in ex-2.1.10 it falls a little under the control level. Those are the same conditions the axis read put back in the band, so the two reads tell one story.

The seeds of a light anchor share an axis and otherwise vary as controls do. The seeds of a heavier or longer anchor share an arrangement beyond the axis, and they reproduce it more closely than controls reproduce theirs.



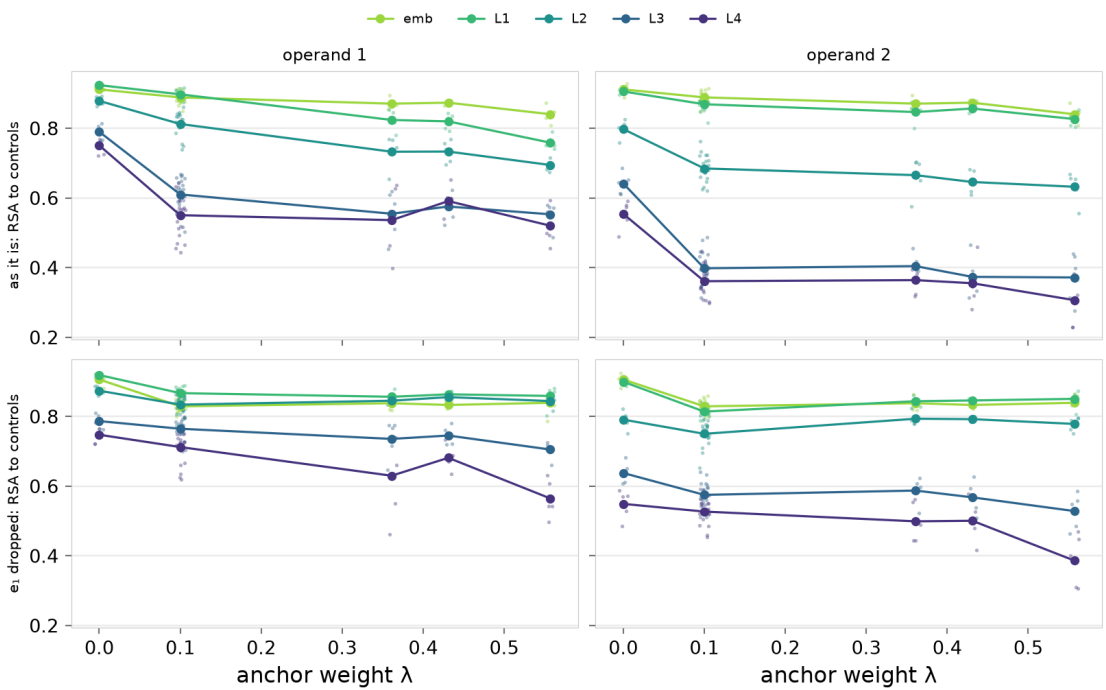
RSA within each condition, at operand 1. Each small dot is one pair of runs of the same condition, the larger mark the mean over pairs, and the grey strip the range of control pairs (the same strip as in the figures above).
Top row: the geometry as it is. Bottom row: e_1 dropped from every run.

Dose

What we expected. If the difference scales with the anchor, the three heavier survey points should sit further from the controls than the recipe, in order of λ .

What we saw. In the full read the drop at the last block is a step rather than a slope. The recipe at $\lambda=0.1$ is already most of the way down, and the heavier points scatter around it. At the earlier blocks the conditions do fall in order of λ .

With e_1 dropped, the last block is graded too: the recipe returns to the band, `t48` nearly reaches it, and `t12` and `t00` stay further off. So the anchor axis seems to account for a fixed part of the distance at any λ , while the rest grows with the weight.



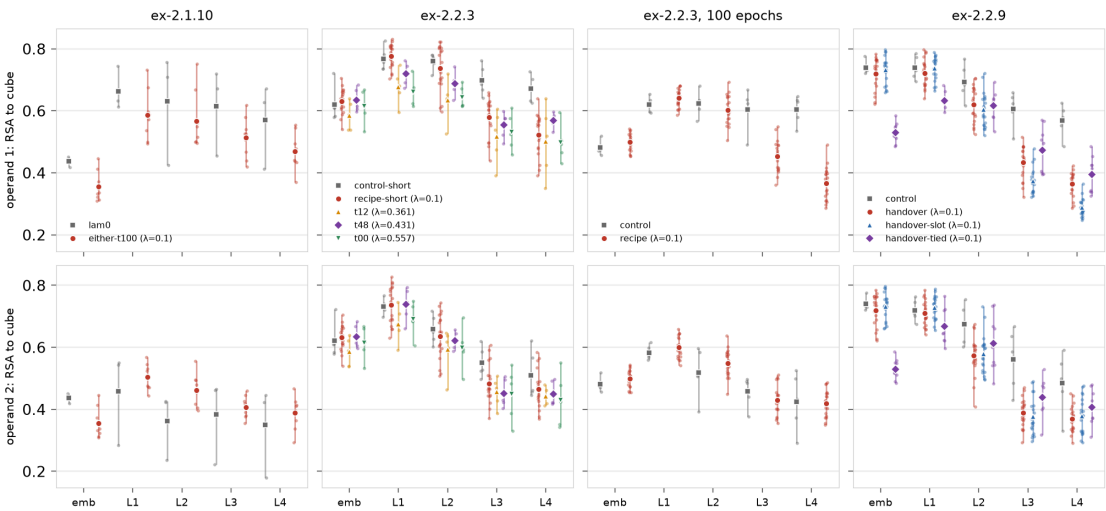
RSA to controls against anchor weight, on ex-2.2.3's short conditions.

Each line is one residual slice (the embedding lightest, the last block darkest), joining the seed means; the dots behind them are seeds. $\lambda=0$ is the control, read against the other controls. Top row: the geometry as it is. Bottom row: e_1 dropped from every run.

Against the colour cube

A control does not arrange the colours as the RGB cube does either, but it comes closer than an anchored run does. This read correlates each run's colour distances with the straight-line distances between the same colours in RGB.

At the embedding and the first block, the conditions are alike, with the embedding of the tied readout as the one exception. From the second block on, the anchored conditions fall further. At the last block on the handover grammar, a control correlates with the cube at about a half and the handover arms at about a third. The heavier ex-2.2.3 anchors and `handover-tied` sit between.



RSA between each run's colour geometry and the RGB cube itself, per slice and site. The cube's distances are the Euclidean distances between the 216 grid colours in RGB. Each dot is one seed, the larger mark the seed mean. A run that arranges the colours as the cube does scores 1.

Checks on the anchor axis

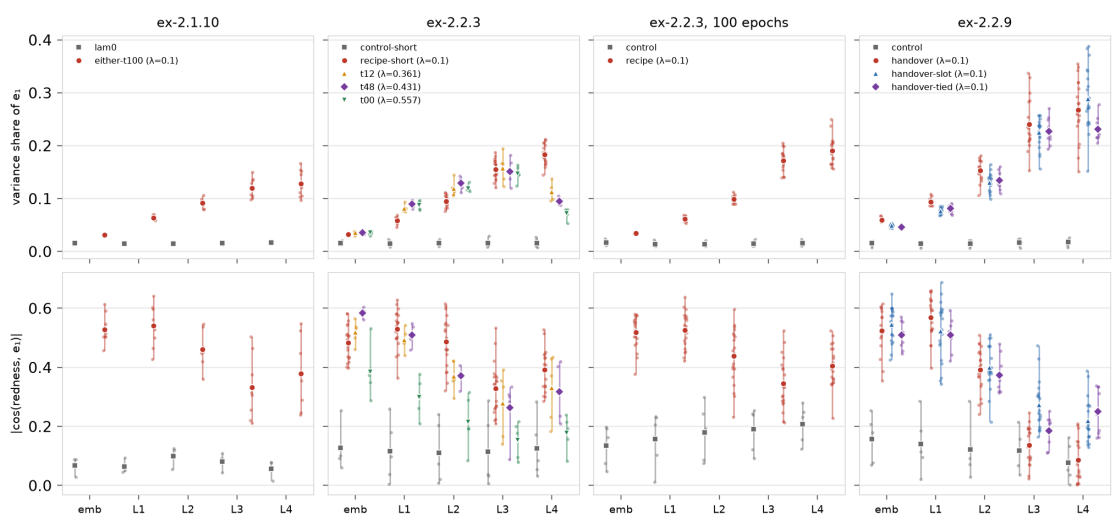
Two reads of e_1 itself, so that the `e1 dropped` rows above can be interpreted.

The first is how much of a run's variance sits along e_1 . In an anchored run it is twice the control level at the embedding, and it grows with depth, reaching ten to eighteen times the control level at the last block. For a control the share is one part in 64, the same as any other coordinate. So the axis is where anchoring put its variance, as intended.

At the last block on the handover grammar, dropping e_1 removes up to a quarter of an anchored run's total variance, against under two percent for a control.

The second is how closely e_1 lines up with the direction a ridge probe reads *red* from. For anchored runs the cosine between them is about a half at the embedding and the first block, and it falls with depth; for controls it is about a tenth. So the probe finds red mostly along e_1 early and less so late, which is how ex-2.1.12 read it.

Together these say what the `e1 dropped` rows take away: a coordinate that is a large part of the deep variance of an anchored run and only part of where its *red* lives. The coordinate is largest on the handover grammar, and even there the last-block gap remains once it is gone.

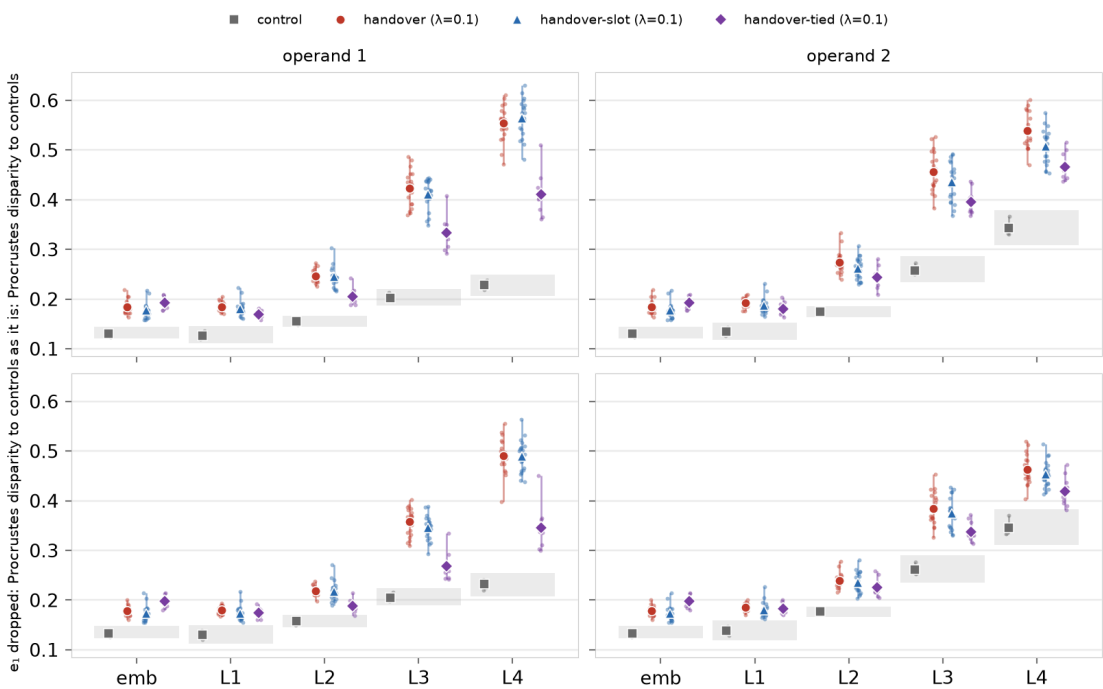


The anchor axis at operand 1. Top: the share of the states' total variance along e_1 . Bottom: the absolute cosine between e_1 and the direction a ridge fit (to the grading target `sim_to_red`, power 1.5) reads red from. Each dot is one seed, the larger mark the seed mean.

A second statistic: Procrustes

RSA works from distances, so it cannot see a rotation. Procrustes disparity takes a different route to the same question. It finds the rotation, reflection, and scale that best map the states of one run onto those of another, then reports what is left over. So it also compares shape, but through the coordinates rather than the distances.

It agrees with RSA on every read. Anchored runs sit above the control band from the second block on, which here means further from the controls. Dropping e_1 changes little on the handover grammar, and the tied readout keeps the most. The figure shows the handover grammar; the other groups are in the store.



Procrustes disparity to controls for ex-2.2.9, per slice and site. Procrustes disparity is the residual after the best rotation and scale between one run and each of the group's control runs (lower is more alike); each small dot is one seed's mean over the controls, the larger mark the seed mean, and the bar the seed range. The grey strip is the range of control-against-control pairs. Top row: the geometry as it is. Bottom row: the anchor axis e_1 dropped from every run first.

What we make of it

The D2.1 post claims that anchoring leaves the rest of the representation to form as it would have. For a light anchor and a short run, that is what we see here once the axis is set aside: the ex-2.2.3 recipe at 50 epochs is a control geometry plus e_1 , and the D2.1 recipe is close to that.

It is not what we see for a heavier anchor, a longer run, or the handover grammar. There the colours are arranged differently beyond the axis, and the difference grows with depth, with training, and with the anchor weight. Seeds reproduce that arrangement more closely than un-anchored seeds reproduce theirs, and it is less like the RGB cube.

Three things this read does not say. It does not say the task is affected: every one of these conditions matched its control on held-out exact match in its own report. It does not say the other channels are lost: ex-2.1.12 found green and blue as decodable as before. And it does not say where in training the two geometries part company, since we only read final checkpoints.

What it says is that the *arrangement* of the colours changes, which is what a whole-geometry statistic measures and a per-channel probe does not.

Two readings are open. The first is that the anchor term reshapes the deep geometry around the axis it is given, so the rest of the space organises relative to *red* rather than as an un-anchored model would have it. The falling RSA against the cube and the rising agreement within a condition both fit that.

The second is that the anchored geometry is the un-anchored one, stretched along e_1 and sheared, in a way that dropping a single coordinate cannot undo. The recipe at 50 epochs fits the stretching part, since dropping e_1 puts it back in the band; whether the handover arms fit the shearing part is open. Telling the two readings apart is a preregistered question, and the statistics here are cheap enough to run at every checkpoint of a training run.

Next. A preregistered experiment with one hypothesis: with e_1 dropped, the last-block RSA of an anchored run to the controls is inside the control band. The stored runs already say it holds for the six-op recipe at 50 epochs and misses for the handover grammar. So the experiment should read the handover grammar through training, at every saved checkpoint, and add an arm at a lower λ .

Two exploratory reads to carry with it: comparing each run against a *redness-only* set of distances, which would say whether the anchored arrangement organises by red; and the same figures at the answer position, where the side-effects of an intervention would land.

Method

Prompts. Every ordered pair of the 216 grid colours runs as the three-token prompt `a op b`, with `op` the grammar's `mix` where it has one and `+` on the D2.1 grammar. The residual stream is read at every slice above positions 0 and 2. The state above position 0 is the same for every `b` (checked to $1e-5$ in every run); the state above position 2 is averaged over `a`.

Dissimilarity. From a `216 × 64` state matrix we take the pairwise Euclidean distances and keep the upper triangle. The states lie on the nGPT hypersphere, so this is the chord distance, which falls as the cosine rises. RSA is then the Pearson correlation between the upper triangles of two runs.

The anchor axis. The `e1 dropped` variant deletes coordinate 0 of every run's states before the distances are taken, and the states are not renormalised. The redness direction in the checks comes from a ridge fit (`l2 = 1e-2`, on centred states) from the states of a run to the grading target `sim_to_red(GRID_RGB, power=1.5)`, normalised to a unit vector.

Procrustes. SciPy's `procrustes`. It centres both matrices and scales them to unit Frobenius norm, finds the best orthogonal map from one to the other, and reports the sum of squared residuals as the disparity. Two geometries that differ only by a rotation, a reflection, or a scale score 0.

Where the data is. The run table is under the `metrics` ref of `reports/m2/geometry-rsa`, and every pairwise matrix under the `arrays` ref. The raw states of each run are under `states/{experiment}/{label}`, so a follow-up can read them without a forward pass. This pass ran on the dev storage pair, reading the source checkpoints from production. It is to be re-run on production before anything here is quoted.

-
1. Those matrices of distances are *representational dissimilarity matrices*.

Correlating two of them asks whether both runs find the same colours near each other and far apart, without requiring them to use the same coordinates. The measure does not change if you rotate the state space, so it is blind to *where* the anchor put red. ↩